

Android 第八章第三节教学讲义（高级控件核心实战：购物车终极升级）

各位同学，今天咱们聚焦三个核心知识点：下拉控件 Spinner、万能列表 RecyclerView，以及如何用它们对购物车进行终极升级。这些控件是 Android 开发的 "必备技能"，尤其是 RecyclerView，灵活又高效，几乎所有复杂列表都离不开它。咱们结合购物车案例，边学边做，保证小白也能上手！

一、下拉控件 Spinner：从基础到实战

Spinner 就像一个 "下拉选择框"，适合从固定选项中选择（比如购物车中的 "排序方式" 选择）。

1. Spinner 基础用法（复习 + 实战）

步骤 1：布局添加 Spinner

```
<Spinner
    android:id="@+id/spinner_sort"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:hint="排序方式"
    android:padding="10dp" />
```

步骤 2：准备数据源（排序选项）

在 Java 代码中直接定义数组（也可以用 strings.xml）：

```
// 排序选项：默认、价格从低到高、价格从高到低
private String[] sortOptions = {"默认排序", "价格 ↓", "价格 ↑"};
```

步骤 3：设置适配器并绑定数据

```
Spinner spinner = findViewById(R.id.spinner_sort);
// 创建适配器（上下文、选项样式、数据源）
ArrayAdapter<String> adapter = new ArrayAdapter<>(
    this,
    android.R.layout.simple_spinner_dropdown_item, // 下拉选项样式
    sortOptions
);
// 设置默认显示样式
adapter.setDropDownViewResource(android.R.layout.simple_spinner_item);
// 绑定适配器
spinner.setAdapter(adapter);
```

步骤 4：监听选择事件（核心：实现排序功能）

```
spinner.setOnItemSelectedListener(new AdapterView.OnItemSelectedListener() {
    @Override
    public void onItemSelected(AdapterView<?> parent, View view, int position, long id) {
        String selected = sortOptions[position];
        // 根据选择的排序方式刷新列表
        sortGoods(position);
        Toast.makeText(MainActivity.this, "已选择：" + selected, Toast.LENGTH_SHORT).show();
    }
    @Override
    public void onNothingSelected(AdapterView<?> parent) {}
});
// 排序逻辑
private void sortGoods(int sortType) {
    switch (sortType) {
        case 0: // 默认排序（按添加顺序）
            // 恢复原始顺序（假设原始数据存在originalList中）
            goodsList.clear();
            goodsList.addAll(originalList);
            break;
        case 1: // 价格从低到高
            Collections.sort(goodsList, (g1, g2) -> (int) (g1.getPrice() - g2.getPrice()));
            break;
        case 2: // 价格从高到低
            Collections.sort(goodsList, (g1, g2) -> (int) (g2.getPrice() - g1.getPrice()));
            break;
    }
}
```

```
}  
// 刷新列表  
adapter.notifyDataSetChanged();  
}
```

二、列表类控件 RecyclerView：万能列表的全方位解析

RecyclerView 是目前 Android 开发中最推荐的列表控件，比 ListView 更灵活（支持线性、网格、瀑布流布局），性能更好（强制复用机制）。

1. RecyclerView 核心三要素

- **LayoutManager**：控制列表布局方式（线性、网格等）；
- **Adapter**：将数据绑定到列表项；
- **ViewHolder**：缓存列表项控件，优化性能。

2. 实战：用 RecyclerView 展示购物车商品

步骤 1：添加依赖（已添加可忽略）

```
implementation 'androidx.recyclerview:recyclerview:1.3.2'
```

步骤 2：布局添加 RecyclerView

```
<androidx.recyclerview.widget.RecyclerView  
    android:id="@+id/recycler_cart"  
    android:layout_width="match_parent"  
    android:layout_height="0dp"  
    android:layout_weight="1" /> <!-- 占满剩余空间 -->
```

步骤 3：创建购物车列表项布局（item_cart.xml）

包含复选框、商品图片、名称、价格、数量加减按钮：

```
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="120dp"
    android:gravity="center_vertical"
    android:padding="10dp"
    android:descendantFocusability="blocksDescendants"> <!-- 解决子控件焦点冲突 -->
    <!-- 选择复选框 -->
    <CheckBox
        android:id="@+id/cb_select"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content" />
    <!-- 商品图片 -->
    <ImageView
        android:id="@+id/iv_goods"
        android:layout_width="100dp"
        android:layout_height="100dp"
        android:layout_marginStart="10dp"
        android:src="@mipmap/ic_launcher" />
    <!-- 商品信息 -->
    <LinearLayout
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginStart="10dp"
        android:layout_weight="1"
        android:orientation="vertical">
        <TextView
            android:id="@+id/tv_name"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:text="商品名称"
            android:textSize="16sp" />
        <TextView
            android:id="@+id/tv_price"
            android:layout_width="wrap_content"
            android:layout_height="wrap_content"
            android:layout_marginTop="10dp"
            android:text="¥99.00"
            android:textColor="#FF0000"
            android:textSize="15sp" />
    </LinearLayout>
```

```
<!-- 数量加减 -->
<LinearLayout
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:orientation="horizontal">
    <Button
        android:id="@+id/btn_minus"
        android:layout_width="30dp"
        android:layout_height="30dp"
        android:text="-" />
    <TextView
        android:id="@+id/tv_count"
        android:layout_width="30dp"
        android:layout_height="30dp"
        android:gravity="center"
        android:text="1" />
    <Button
        android:id="@+id/btn_plus"
        android:layout_width="30dp"
        android:layout_height="30dp"
        android:text="+" />
</LinearLayout>
</LinearLayout>
```

步骤 4：创建商品实体类（CartGoods.java）

```
public class CartGoods {
    private int id; // 商品ID
    private String name; // 商品名称
    private double price; // 单价
    private int count; // 数量
    private boolean isSelected; // 是否选中
    // 构造方法
    public CartGoods(int id, String name, double price, int count, boolean isSelected) {
        this.id = id;
        this.name = name;
        this.price = price;
        this.count = count;
        this.isSelected = isSelected;
    }
}
```

```

}
// getter和setter（必须有，用于修改数据）
public int getId() { return id; }
public String getName() { return name; }
public double getPrice() { return price; }
public int getCount() { return count; }
public void setCount(int count) { this.count = count; }
public boolean isSelected() { return isSelected; }
public void setSelected(boolean selected) { isSelected = selected; }
}

```

步骤 5：创建 RecyclerView 适配器（核心）

```

public class CartAdapter extends RecyclerView.Adapter<CartAdapter.CartViewHolder> {
    private Context mContext;
    private List<CartGoods> mGoodsList; // 购物车数据
    private OnCartChangeListener mListener; // 数据变化监听器（回调给Activity）
    // 构造方法：接收数据和监听器
    public CartAdapter(Context context, List<CartGoods> goodsList, OnCartChangeListener
listener) {
        mContext = context;
        mGoodsList = goodsList;
        mListener = listener;
    }
    // 1. 创建ViewHolder（加载列表项布局）
    @Override
    public CartViewHolder onCreateViewHolder(ViewGroup parent, int viewType) {
        View view = LayoutInflater.from(mContext)
            .inflate(R.layout.item_cart, parent, false);
        return new CartViewHolder(view);
    }
    // 2. 绑定数据（设置控件内容和事件）
    @Override
    public void onBindViewHolder(CartViewHolder holder, int position) {
        CartGoods goods = mGoodsList.get(position);
        // 绑定数据
        holder.cbSelect.setChecked(goods.isSelected());
        holder.tvName.setText(goods.getName());
        holder.tvPrice.setText("¥" + goods.getPrice());
    }
}

```

```

holder.tvCount.setText(String.valueOf(goods.getCount()));
// 3. 复选框点击事件 (修改选中状态)
holder.cbSelect.setOnCheckedChangeListener((buttonView, isChecked) -> {
    goods.setSelected(isChecked);
    mListener.onSelectChange(); // 通知Activity更新全选和总价
});
// 4. 减号按钮 (数量-1, 最少1)
holder.btnMinus.setOnClickListener(v -> {
    int count = goods.getCount();
    if (count > 1) {
        goods.setCount(count - 1);
        holder.tvCount.setText(String.valueOf(count - 1));
        mListener.onCountChange(); // 通知Activity更新总价
    }
});
// 5. 加号按钮 (数量+1)
holder.btnPlus.setOnClickListener(v -> {
    int count = goods.getCount();
    goods.setCount(count + 1);
    holder.tvCount.setText(String.valueOf(count + 1));
    mListener.onCountChange(); // 通知Activity更新总价
});
}
// 6. 返回数据总数
@Override
public int getItemCount() {
    return mGoodsList.size();
}
// ViewHolder: 缓存控件
public static class CartViewHolder extends RecyclerView.ViewHolder {
    CheckBox cbSelect;
    ImageView ivGoods;
    TextView tvName, tvPrice, tvCount;
    Button btnMinus, btnPlus;
    public CartViewHolder(View itemView) {
        super(itemView);
        cbSelect = itemView.findViewById(R.id.cb_select);
        ivGoods = itemView.findViewById(R.id.iv_goods);
        tvName = itemView.findViewById(R.id.tv_name);
        tvPrice = itemView.findViewById(R.id.tv_price);
        tvCount = itemView.findViewById(R.id.tv_count);
    }
}

```

```

        btnMinus = itemView.findViewById(R.id.btn_minus);
        btnPlus = itemView.findViewById(R.id.btn_plus);
    }
}
// 定义回调接口（通知Activity数据变化）
public interface OnCartChangeListener {
    void onSelectChange(); // 选中状态变化
    void onCountChange(); // 数量变化
}
}

```

3. 在 Activity 中初始化 RecyclerView 并实现功能

步骤 1：布局添加底部操作栏（全选、删除、总价、结算）

```

<LinearLayout
    android:layout_width="match_parent"
    android:layout_height="50dp"
    android:background="#FFFFFF"
    android:gravity="center_vertical"
    android:padding="0 10dp">
    <CheckBox
        android:id="@+id/cb_all_select"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="全选" />
    <Button
        android:id="@+id/btn_delete"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginStart="10dp"
        android:text="删除选中" />
    <TextView
        android:id="@+id/tv_total"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginStart="auto"
        android:text="合计：¥0.00"
    />
</LinearLayout>

```



```
        android:textColor="#FF0000" />
<Button
    android:id="@+id/btn_pay"
    android:layout_width="wrap_content"
    android:layout_height="wrap_content"
    android:layout_marginStart="10dp"
    android:background="#FF0000"
    android:text="结算"
    android:textColor="#FFFFFF" />
</LinearLayout>
```

步骤 2: Activity 核心代码

```
public class CartActivity extends AppCompatActivity implements CartAdapter-
.OnCartChangeListener {
    private RecyclerView recyclerCart;
    private CartAdapter adapter;
    private List<CartGoods> goodsList;
    private CheckBox cbAllSelect;
    private TextView tvTotal;
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_cart);
        // 初始化控件
        recyclerCart = findViewById(R.id.recycler_cart);
        cbAllSelect = findViewById(R.id.cb_all_select);
        tvTotal = findViewById(R.id.tv_total);
        // 1. 准备购物车数据
        initData();
        // 2. 设置RecyclerView
        recyclerCart.setLayoutManager(new LinearLayoutManager(this)); // 线性布局
        adapter = new CartAdapter(this, goodsList, this); // 传入监听器
        recyclerCart.setAdapter(adapter);
        // 3. 全选按钮事件
        cbAllSelect.setOnCheckedChangeListener((buttonView, isChecked) -> {
            for (CartGoods goods : goodsList) {
                goods.setSelected(isChecked);
            }
        })
    }
}
```

```
        adapter.notifyDataSetChanged(); // 刷新列表
        updateTotalPrice(); // 更新总价
    });
    // 4. 删除选中按钮事件
    findViewById(R.id.btn_delete).setOnClickListener(v -> {
        List<CartGoods> toDelete = new ArrayList<>();
        for (CartGoods goods : goodsList) {
            if (goods.isSelected()) {
                toDelete.add(goods);
            }
        }
        goodsList.removeAll(toDelete);
        adapter.notifyDataSetChanged();
        updateTotalPrice();
        checkAllSelected(); // 重新检查全选状态
    });
    // 5. 结算按钮事件
    findViewById(R.id.btn_pay).setOnClickListener(v -> {
        int selectedCount = 0;
        for (CartGoods goods : goodsList) {
            if (goods.isSelected()) {
                selectedCount++;
            }
        }
        if (selectedCount == 0) {
            Toast.makeText(this, "请选择商品", Toast.LENGTH_SHORT).show();
        } else {
            Toast.makeText(this, "结算成功！共" + selectedCount + "件商品", Toast.LENGTH_SHORT).show();
        }
    });
}
// 初始化购物车数据
private void initData() {
    goodsList = new ArrayList<>();
    goodsList.add(new CartGoods(1, "小米手机", 3999, 1, false));
    goodsList.add(new CartGoods(2, "华为平板", 2999, 1, true));
    goodsList.add(new CartGoods(3, "苹果耳机", 1299, 2, false));
}
// 更新总价（实现接口方法）
@Override
```

```
public void onCountChange() {
    updateTotalPrice();
}
// 选中状态变化时更新全选和总价（实现接口方法）
@Override
public void onSelectChange() {
    checkAllSelected(); // 检查是否全选
    updateTotalPrice(); // 更新总价
}
// 计算总价
private void updateTotalPrice() {
    double total = 0;
    for (CartGoods goods : goodsList) {
        if (goods.isSelected()) {
            total += goods.getPrice() * goods.getCount();
        }
    }
    tvTotal.setText("合计: ¥" + total);
}
// 检查是否全选
private void checkAllSelected() {
    boolean isAllSelected = true;
    for (CartGoods goods : goodsList) {
        if (!goods.isSelected()) {
            isAllSelected = false;
            break;
        }
    }
    cbAllSelect.setChecked(isAllSelected);
}
}
```

三、案例：购物车终极升级功能总结

通过 Spinner 和 RecyclerView，我们的购物车实现了以下核心功能：

1. **商品列表展示**：用 RecyclerView 展示商品图片、名称、价格、数量；
2. **选择功能**：单个商品选择、全选 / 取消全选；
3. **数量调整**：加减按钮修改商品数量；

4. **价格计算**：自动计算选中商品的总价；
5. **删除功能**：删除选中的商品；
6. **排序功能**：用 Spinner 选择按价格排序（从高到低 / 从低到高）。

总结

这节课咱们掌握了两个核心控件和一个综合案例：

1. **Spinner**：通过适配器绑定数据，监听选择事件实现排序等功能；
2. **RecyclerView**：
 - 三要素：LayoutManager（布局）、Adapter（数据绑定）、ViewHolder（缓存）；
 - 自定义 Adapter 时，通过接口回调将列表项的事件（选择、数量变化）通知给 Activity；
3. **购物车升级**：整合 Spinner 和 RecyclerView，实现选择、数量调整、总价计算、删除、排序等完整功能。

RecyclerView 是 Android 开发中最常用的控件之一，一定要多练习！课后可以给购物车添加 "商品图片加载"（从网络或本地加载真实图片）和 "持久化存储"（用 Room 保存购物车数据，下次打开不丢失）功能，进一步巩固所学。有问题随时问～

（注：文档部分内容可能由 AI 生成）